

7. Princip činnosti počítače (řetězené zpracování instrukcí, RISC, CISC)

7. Princip činnosti počítače — pipelining, RISC, CISC

SZZ okruh 7 (FIT BUT). CPU je rozdělen na **stupně**, které mohou současně zpracovávat různé instrukce v různém stádiu dokončení.

Shrnutí

Zřetězené zpracování (pipelining)

- Stupně: **Fetch** → **Decode** → **Execute** → **Memory Access** → **Write Back**.
- Ideálně se po každém taktu dokončí jedna instrukce → **CPI = 1**.
- Zrychlení → $N \cdot k \cdot t / ((k + N - 1) \cdot (t + d))$, kde N = počet instrukcí, k = počet stupňů, t = zpoždění stupně, d = zpoždění registru.

[[media/szz-07/media/image3.png]] Zřetězené zpracování — instrukce v různých stupních linky současně.

Konflikty (hazardy)

- **Strukturální** — HW neumožní dvě akce naráz (např. dvojí čtení z paměti).
- **Datové** — instrukce potřebuje výsledek nedokončené předchozí instrukce.
 - Typy: **RAW** (read after write), **WAW**, **WAR**; **RAR není hazard**.
 - Řešení: **forwarding** / **bypassing** (datové cesty mezi stupni); u LOAD+ADD se přesto musí čekat.
- **Řídící** — skok mění PC. Řešení: **zpožděný skok** (překladač vyplní nezávislými instrukcemi) a **prediktor skoku** + **BTB** (branch target buffer); ~80 % skoků se provede, predikce statická/dynamická.

RISC vs. CISC

	RISC	CISC
Instrukční sada	redukováná, jeden adresovací režim	složitá, mnoho režimů
Délka instrukce	stejná	proměnná
Přístup do paměti	jen LOAD/STORE	memory-to-memory ap.
CPI	→ 1	> 1
Registry	mnoho (dominují na čipu)	málo (často jen střadač)
Příklady	ARM, RISC-V, Apple Silicon	Intel x86

Souvislosti

Latence paměti řeší [[okruhy/04-hierarchie-pameti|cache]]; architektura MCU a volba RISC/CISC viz [[okruhy/05-vestavene-systemy]]; přerušení jako řídicí hazard viz [[okruhy/06-periferie-preruseni-dma-sbernice]]; sčítačky v ALU viz [[okruhy/02-kombinacni-logicke-obvody]].

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Základní činnost CPU □ [[#Zřetěžené zpracování (pipelining)]] - *Cyklus fetch-decode-execute?* □ CPU načte instrukci (fetch), dekóduje (decode), provede (execute), přístup do paměti (memory) a zápis výsledku (write back).

Pipelining □ [[#Zřetěžené zpracování (pipelining)]] - *Fáze pipeline?* □ F □ D □ E □ M □ W. - *Proč pipeline, CPI?* □ Více instrukcí současně v různých fázích □ ideálně CPI = 1; reálně > 1 kvůli hazardům.

Hazardy □ [[#Konflikty (hazardy)]] - *Typy?* □ strukturální (sdílený zdroj), datové (RAW / WAR / WAW; RAR není hazard), řídicí (skoky). - *Řešení?* □ forwarding/bypassing, stall/NOP, přeskládání instrukcí, predikce skoku (BTB).

RISC vs. CISC □ [[#RISC vs. CISC]] - *Rozdíly, proč RISC pro pipeline?* □ RISC: málo stejně dlouhých instrukcí, do paměti jen LOAD/STORE, mnoho registrů, CPI □ 1 □ snadné zřetězení. CISC: složité instrukce proměnné délky. - *Příklady?* □ RISC: ARM, RISC-V, Apple Silicon; CISC: Intel x86.

Plné znění (ke studiu)

[[media/szz-07/media/image1.png]]

Zřetěžené zpracování instrukcí (pipelining)

Procesor je velmi složitý obvod a provedení instrukce se typicky děje v **různých** jeho částech. Proto jsou CPU rozděleny na **stupně**, každý stupeň může **současně** řešit jinou část instrukce. To znamená, že lze současně provádět výpočet více instrukcí, každou v **jiném stavu dokončení**. Mezi stupni CPU přechází instrukce s **taktem** hodin, ideálně lze díky **zřetěženému zpracování (pipelining)** neustále využívat **všechny** stupně CPU a po **každém taktu** dokončit jednu instrukci (CPI = 1 cycles per instruction). Např. dnešní Intel procesory obsahují 14 stupňovou linku. [[media/szz-07/media/image2.png]]

[[media/szz-07/media/image3.png]]

Stavy provádění, ve kterých se instrukce může nacházet vypadají:

- **Fetch** (F) - načtení instrukce,
- **Decode** (D) - dekódování instrukce,
- **Execute** (E) - vykonání instrukce,
- **Memory Access** (M) - čtení z paměti,
- **Write Back** (W) - zápis do registru

Zrychlení zřetěžené linky

$$\frac{N * k * t}{(k + N - 1) * (t + d)}$$

kde:

- **N** je počet instrukcí,
- **k** je počet stupňů zřetězené linky,
- **t** je zpoždění stupně,
- **d** je zpoždění registru (mezipaměti - klopný obvod)

Konflikty (hazardy)

Situace, které je nutné při řetězení instrukcí řešit, jinak by mohl být výsledek výpočtů nesprávný. Často konflikty způsobují **zpomalení** linky (**stall**) linka musí čekat na **dokončení** některé instrukce. Někdy stačí, když překladač změni pořadí **nezávislých** instrukcí pro vyřešení konfliktu. Konflikty dělíme na:

- **Strukturální** – obvodová struktura neumožňuje současné provedení určitých akcí, např. současné čtení dvou hodnot z paměti.
- **Datové** – jsou zapotřebí data z **předcházející** instrukce, která ještě není dokončena (např. u dvou následujících instrukcí využívající stejný register). Instrukce ADD R1,R2,R3 a po ní následující SUB R4,R5,R1 pracují s registrem R1, druhá instrukce by pracovala již s **neplatnou** hodnotou, pokud by problém nebyl řešen. Hloupé řešení je pozastavit linku a počkat na dokončení první instrukce. Chytré ale dražší řešení je doplnit CPU o datové cesty, které umožní předávat mezivýsledky (**forwarding** nebo **bypassing**) mezi jednotlivými stupni CPU. Ne vždy to je ale možné, např. u dvojice instrukcí LOAD R1,addr a ADD R3,R1,R2. obsah R1 je získán až na konci fáze Memory Access první instrukce (konec 4. taktu), instrukce ADD jej potřebuje ale už na začátku fáze Execute ve svém 3. taktu (celkově začátek 4. taktu), linka tak musí **čekat**. Datové hazardy dělíme na:
 - **RAW – Read After Write:** 2. instrukce se pokouší číst zdroj před tím, než do něj zapsala 1. instrukce.
 - **WAW – Write After Write:** 2. instrukce se pokouší zapsat operand dříve, než je proveden zápis 1. instrukcí.
 - **WAR – Write After Read:** 2. instrukce se pokouší zapsat operand dříve, než je 1. instrukcí přečten.
 - **NENÍ HAZARD: RAR – Read After Read:** obě pouze čtou, tedy nemůže dojít k chybě (hazardu)
- **Řídící** – skoková instrukce **mění** obsah PC (program counter, ukazatel na následující instrukci), nebo jiné. U podmíněných skoků procesor nezná adresu instrukce následující po skoku, nemůže tak načítat (fáze Fetch) další instrukce a linka musí být pozastavena. Pozastavení lze ale předejít v případech:
 - V programu se vyskytují jiné nezávislé instrukce, kterými lze vyplnit prostor mezi **získáním** adresy a skokem, jde o **zpožděný skok**. Změnu pořadí instrukcí provádí překladač, musí zajistit, aby nedošlo ke změně výsledku.
 - Zavést specializované obvody (**prediktor skoku a paměť cílové adresy skoku - BTB**), které odhadnou, zda **provést/neprovést** skok a adresu skoku. Dle odhadu se začínou načítat instrukce dle adresy z **BTB (branch target buffer - cache**, kterou CPU postupně naplňuje a aktualizuje) a linka se v případě správného odhadu **nepozastaví**. Pokud byl odhad **špatný**, je provádění těchto instrukcí předčasně ukončeno a začíná se s prováděním těch **správných**, to ale způsobí **zpomalení** linky. U moderních procesorů, které mají **delší linky** (10-20 stupňů), je velký požadavek na kvalitní prediktory. Bylo zjištěno, že se provede většinou kolem **80%** skoků. Predikce může být **statická** - určí ji překladač nebo **dynamická**, určuje ji procesor (speciální HW obvod). Nejjednodušší dynamická predikce odhaduje skok na základě toho, jak byl vykonán v předcházejícím běhu.

Ortogonalní instrukční sada

Instrukce fungují se všemi adresovacími mody (instrukce není nucená například mít jako první operand akumulátor).

RISC (Reduced Instruction Set Computer)

- **Redukovaná instrukční sada.**
- Oproti CISC má menší množství instrukcí s **jedním** adresovacím režimem - **register**, načítat data do/z registrů umožňují **pouze** instrukce LOAD a STORE zajišťující přístup do paměti.
- všechny instrukce jsou stejně dlouhé (**stejný počet bitů** v paměti).
- Na zřetěžené lince lze ideálně zajistit dokončení instrukce **každý** takt: **CPI = 1**. To znamená, že každá instrukce musí trvat **stejný počet taktů**.
- Využívá velké množství **registrů**.
- Na čipu **dominují registry** (paměť) oproti obvodům pro dekódování a řízení instrukcí.
- obvykle mají **menší** spotřebu energie,
- ARM, RISC-V.

CISC (Complex Instruction Set Computing)

- **Složitá instrukční sada** - Hlavní myšlenkou procesorů CISC je, že k provádění operací **načítání, vyhodnocení a ukládání** lze použít jedinou instrukci (**memory-to-memory, mem-to-reg, reg-to-mem, reg-to-reg**).
- Mnoho **typů instrukcí** s mnoha variantami a mnoha **adresovacími režimy**.
- Instrukce nemusí mít stejnou délku v bitech - náročnější na dekódování.
- Dle složitosti instrukce navíc mohou trvat různou dobu (počet taktů), hůře se řeší zřetěžené zpracování a **CPI > 1**.
- Komplexní instrukce mohou být na rozdíl od RISC efektivnější, ale složitější na použití.
- Nepoužívá mnoho registrů, někdy pouze **střadač**.
- Na čipu dominuje logika pro **dekódování a řízení** instrukcí před registry.
- Intel x86.

Zdroje

- SZZ okruh 7 — studijní materiály FIT BUT (szz-07.docx). Další obrázky: media/szz-07/.

Navigace: [[index| • Přehled okruhů]] · □ [[okruhy/06-periferie-preruseni-dma-sbernice|6. Připojování periférií]] · další: [[okruhy/08-minimalizace-logických-vyrazu|8. Minimalizace logických výrazů]] □